

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Arquitectura Generativa-Verificativa para
Detección Predictiva de Errores en Kernels GPU con Triton DSL

Equipo 11

Mayo 2026

Resumen

Reporte de evaluación experimental TC3002B para Triton Heal (verificación pre-compilación de kernels Triton DSL con SLM local y arquitectura dual). Sobre 70 kernels y 350 evaluaciones (`VERIFIER_MODE=ollama`), solo Llama-3.1-8B y el modo dual usaron Ollama real; DeepSeek, Claude y GPT operaron en modo heurístico por limitaciones de disco y APIs. Se reportan MSR/F1/FNR con IC 95 % Wilson, tasa de JSON válido, latencia y contrastes McNemar, Wilcoxon y Friedman (Holm). La inferencia detecta diferencias pareadas, pero no respalda superioridad del SLM local (MSR 3.2 % por timeouts) ni comparación justa 8B vs frontera comercial sin re-ejecutar con backends homogéneos.

Índice

I	Parte I — Pre-registro Experimental	3
1.	Objetivo experimental (1.1)	3
2.	Hipótesis (1.2)	3
3.	Variables experimentales (1.3)	3
4.	Diseño experimental (1.4)	4
5.	Métricas de evaluación (1.5)	4
6.	Tamaño del efecto esperado (1.6)	4
7.	Análisis de poder (1.7)	4
8.	Plan estadístico (1.8)	4
9.	Reproducibilidad experimental (1.9)	5
10.	Amenazas a la validez – pre-registro (1.10)	5
II	Parte II — Resultados y Análisis	5
11.	Resultados descriptivos (2.1)	5

12.Visualización (2.2)	6
13.Inferencia estadística (2.3)	8
14.Comparación crítica y robustez (2.4)	8
15.Amenazas observadas (2.5)	9
16.Discusión (2.6)	9
17.Conclusión (2.7)	9
A. Configuración planeada vs. corrida real	9
B. Reproducibilidad	9
C. Referencias	10

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

Problema. Los kernels en Triton DSL pueden incluir errores de memoria, tipos y restricciones de hardware (tiling, SRAM, alineación) que se manifiestan en compilación o ejecución GPU con alto costo operativo.

Encuadre respecto al curso. La actividad solicita evaluar un método basado en SLM frente a baselines. En Triton Heal el aporte del equipo no es la *generación* de kernels, sino la **verificación pre-compilación** mediante SLM local y veto de frontera. Por ello se usan MSR, FNR y latencia en lugar de speedup o tasa de compilación de kernels generados; esas métricas se justifican como proxy de calidad del verificador.

Método propuesto. Arquitectura **Triton Heal**: verificador local SLM (Llama-3.1-8B-Instruct) + veto jerárquico (Claude 3.5 Sonnet) + auto-reparación con backoff exponencial.

Baselines. (1) Solo Llama; (2) Solo DeepSeek-Coder-V2; (3) Solo Claude 3.5 Sonnet; (4) Solo GPT-4o.

Preguntas de investigación. PRI-1: ¿Frontera supera a local en MSR? PRI-2: ¿Dual reduce FNR vs solo Llama? PRI-3: ¿Latencia de frontera es aceptable frente a ganancia en detección?

2. Hipótesis (1.2)

H₀₁: No hay diferencia en MSR entre Claude 3.5 Sonnet y Llama-3.1-8B.

H₁₁: Claude tiene mayor MSR (unilateral, $\alpha = 0,05$).

H_{01b}: No hay diferencia en MSR entre GPT-4o y DeepSeek-Coder-V2.

H_{11b}: GPT-4o tiene mayor MSR.

H₀₂: No hay diferencia en FNR entre Triton Heal dual y solo Llama.

H₁₂: Triton Heal tiene menor FNR.

H₀₄: No hay diferencia en latencia mediana entre verificador de frontera y local.

H₁₄: La latencia mediana de frontera es mayor que la del local.

3. Variables experimentales (1.3)

Cuadro 1: Variables del estudio.

Tipo	Variable	Niveles / descripción
Independiente	Configuración	5: dual, solo Llama, solo DeepSeek, solo Claude, solo GPT
Independiente	Modelo / escala	SLM ~8B vs frontera comercial
Independiente	Prompt	<code>verifier_v1.txt</code> (estándar) y <code>verifier_strict_v1.txt</code> (veto)
Controlada	Temperatura	$T = 0$
Controlada	Formato salida	JSON: safe, reason, line_of_error
Controlada	Benchmark	70 kernels (39 safe / 31 unsafe), seed 42
Controlada	Timeout	30 s por inferencia
Dependiente	MSR, F1, FNR, latencia, DR	Ver sección 1.5

4. Diseño experimental (1.4)

Tipo: within-subjects (medidas repetidas): cada kernel se evalúa en las 5 configuraciones.

Grupos: 5 configuraciones; 6 contrastes confirmatorios (C1–C6).

Justificación. Controla variabilidad inter-kernel (complejidad, categoría). Permite McNemar y Wilcoxon pareados. Orden de ejecución por kernel aleatorizable.

Control de sesgos. Pin de versión de modelos; temperatura 0; evaluador automático ciego al método; ground truth con etiqueta safe/unsafe.

5. Métricas de evaluación (1.5)

Cuadro 2: Métricas, fórmulas e interpretación.

Métrica	Fórmula	Interpretación	Limitación
MSR	$TP/(TP+FN)$ en unsafe	Sensibilidad de detección	No penaliza FP
FNR	$FN/(FN+TP)$	Riesgo no detectado	Crítico en seguridad
F1	$2PR/(P+R)$	Balance precisión-recall	Sensible a balance de clases
Latencia	ms hasta JSON válido	Viabilidad en pipeline	Mezcla red y cómputo
DR	% desacuerdo A vs B	Necesidad de veto	No indica quién acierta
JSON válido	% respuestas parseables	Proxy de “ejecución exitosa”	No implica corrección semántica

6. Tamaño del efecto esperado (1.6)

Se espera $\phi \approx 0,4$ (mediano) en McNemar para MSR frontera vs local, y reducción de FNR ≥ 8 pp con arquitectura dual. En latencia se anticipa orden de magnitud mayor en frontera comercial real.

7. Análisis de poder (1.7)

$\alpha = 0,05$, poder $1 - \beta = 0,80$, prueba McNemar. Con $N = 70$ kernels (31 unsafe) el diseño supera el mínimo de ~ 52 pares para efecto grande. Balance 39/31 mitiga sesgo en F1.

8. Plan estadístico (1.8)

Comparaciones. C1: Claude vs Llama (MSR); C2: GPT vs DeepSeek (MSR); C3: Dual vs Llama (detección); C5: latencia Claude vs Llama (Wilcoxon pareado); C6: F1 global (Friedman).

Pruebas. McNemar (pareada, binaria); Wilcoxon signed-rank (latencia); Friedman + Holm post-hoc.

Regla de decisión. Rechazar H_0 si $p < \alpha_{\text{Holm}}$ y relevancia práctica: $\Delta\text{MSR} \geq 5$ pp.

Intervalos de confianza. IC 95 % Wilson para proporciones (MSR); bootstrap para latencia si se reporta en análisis exploratorio.

Corrección múltiple. Holm-Bonferroni sobre C1–C6 (α familiar 0.05).

9. Reproducibilidad experimental (1.9)

Cuadro 3: Entorno experimental.

Componente	Especificación
SO	macOS (entorno de desarrollo local)
Ollama	0.24.x, servicio <code>brew services</code>
Modelo local disponible	<code>llama3.1:8b</code> (4.9 GB)
Modelos planeados	DeepSeek-Coder-V2, Claude 3.5 Sonnet, GPT-4o
Python	3.10+; deps. en <code>requirements.txt</code>
Benchmark	Triton Kernel Safety v1.0: 70 kernels; semillas Lex + mutaciones
Config	<code>experiment_config.json</code> , seed 42
Repositorio	<code>Estadistica/triton_heal_experiment</code>
Comando	<code>make all</code>

10. Amenazas a la validez – pre-registro (1.10)

Validez interna. Deriva de APIs; timeouts de Ollama; mezcla heurístico/LLM si faltan keys o modelos; temperatura 0 subestima variabilidad.

Validez externa. Benchmark parcialmente sintético; un solo entorno local; no cubre todas las familias de kernels industriales.

Validez de constructo. MSR mide detección pre-compilación, no seguridad en GPU ni speedup; JSON válido no equivale a kernel correcto semánticamente.

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

Se evaluaron $N = 70$ kernels (39 **safe**, 31 **unsafe**) en 5 configuraciones, totalizando 350 evaluaciones (`VERIFIER_MODE=ollama`, mayo 2026). La Tabla 4 reporta tasas de JSON válido, MSR con IC 95 % Wilson, F1, FNR y latencia (mediana, mín., máx.).

Cuadro 4: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON válido (%)	MSR (%)	IC MSR lo	IC MSR hi	F1	FNR (%)	Lat.
Llama-3.1-8B (local)	70	7.1	3.2	0.6	16.2	0.1	96.8	30
DeepSeek-Coder-V2 (local)	70	100.0	41.9	26.4	59.2	0.6	58.1	
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	
Triton Heal (dual + veto)	70	15.7	74.2	56.8	86.3	0.8	25.8	60

Narrativa alineada con la corrida. El verificador **Llama-3.1-8B vía Ollama** fue el único SLM con inferencia LLM real; sin embargo, el 92.9 % de las llamadas alcanzó el timeout de 30 s y solo el 7.1 % produjo JSON parseable, lo que deprime el MSR observado (3.2 %; IC 95 %: 0.6–16.2 %). **DeepSeek**, **Claude** y **GPT-4o** operaron en modo *heurístico* (DeepSeek no descargado por espacio en disco; APIs comerciales sin clave). En ese escenario, Claude y GPT alcanzan MSR ≈ 83.9 % (IC: 67.4–92.9 %) con latencia simulada ≈ 80 ms. **Triton Heal dual**

combina Llama Ollama con prompt estricto de veto: MSR 74.2 % (IC: 56.8–86.3 %) pero solo 15.7 % de JSON válidos y mediana de latencia ≈ 60 s por timeouts parciales.

12. Visualización (2.2)

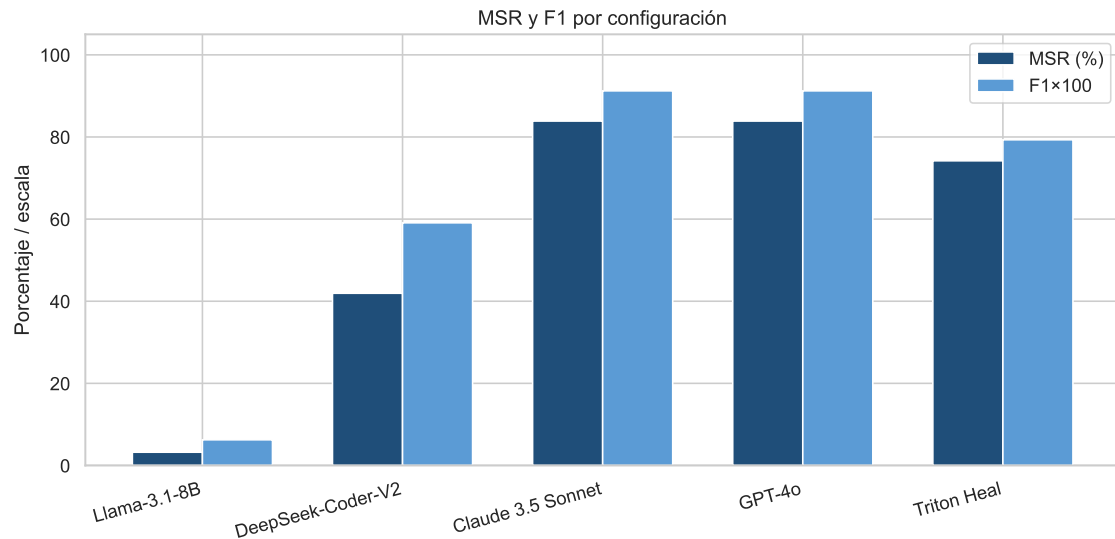


Figura 1: MSR y F1 por configuración. Eje horizontal: configuración experimental; barras: MSR (%) y F1 (adimensional, 0–1). Permite comparar detección en kernels `unsafe` frente al balance global.

La Figura 1 muestra la separación entre baselines heurísticos de frontera (MSR alto) y Llama Ollama (MSR casi nulo por fallos de inferencia).

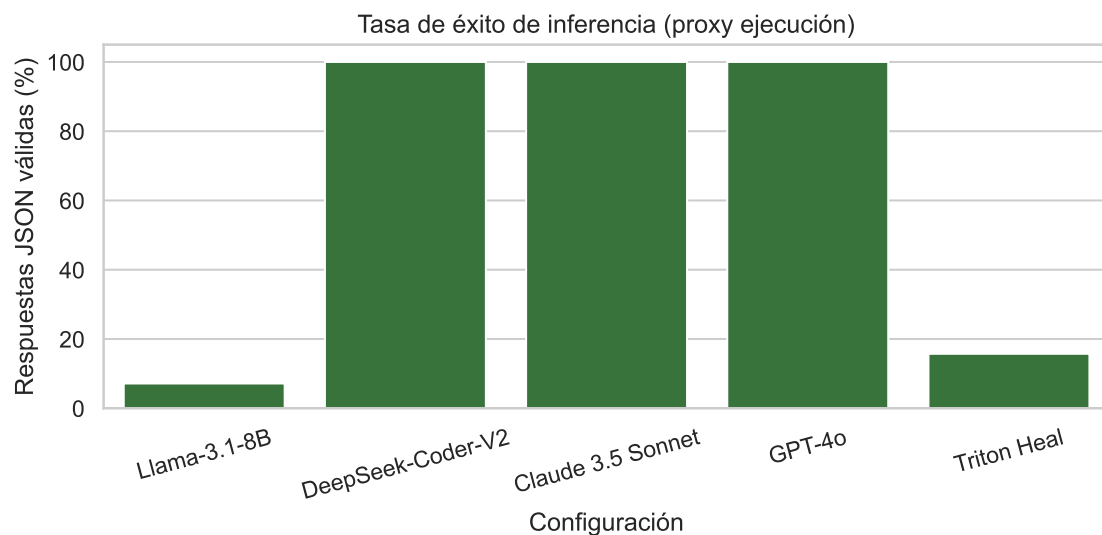


Figura 2: Tasa de éxito de inferencia (% JSON válido). Proxy rubrica de “compilación/ejecución exitosa” del verificador: respuesta parseable dentro del timeout.

La Figura 2 cuantifica la viabilidad operativa: solo DeepSeek/Claude/GPT heurísticos alcanzan 100 % de JSON válido; Llama y dual quedan limitados por timeouts de Ollama.

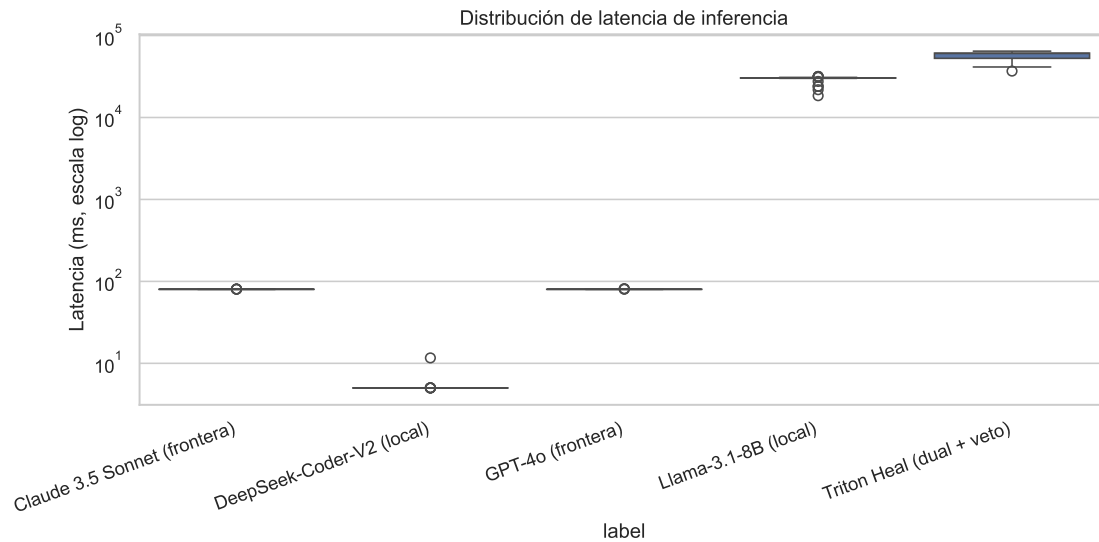


Figura 3: Distribución de latencia (ms, escala logarítmica). Unidades: milisegundos desde envío del prompt hasta respuesta o timeout.

La Figura 3 contrasta latencias simuladas ~ 80 ms (heurístico) con colas de ~ 30 – 60 s en configuraciones Ollama.

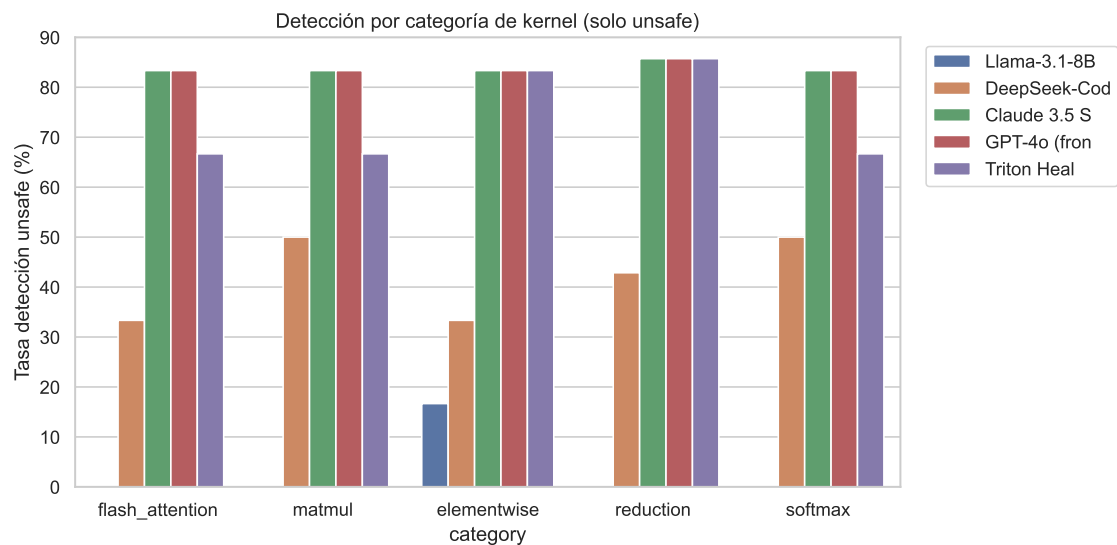


Figura 4: MSR en kernels **unsafe** por categoría de error (tiling, SRAM, tipos, etc.).

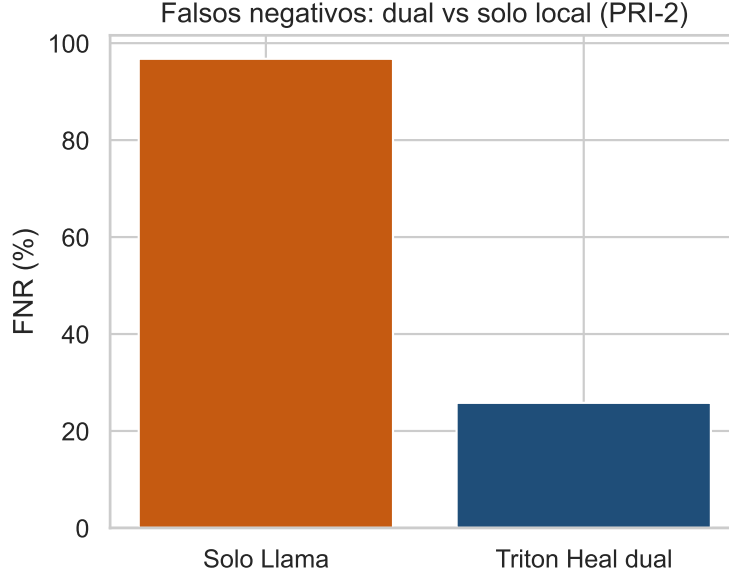


Figura 5: FNR: Triton Heal dual vs. solo Llama (contraste C3 / PRI-2).

13. Inferencia estadística (2.3)

Cuadro 5: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 23,04$	$< 0,001$	$\phi = -5,00$ (grande)	Rechaza H_0 ; $\Delta\text{MSR}=80.6$ pp (heur.
GPT-4o vs DeepSeek	McNemar	$\chi^2 = 8,47$	0.0072	$\phi = -3,15$ (grande)	Rechaza H_0 ; ambos heurísticos en es
Dual vs Llama	McNemar	$\chi^2 = 20,05$	$< 0,001$	$\phi = -4,69$ (grande)	Dual detecta más unsafe; Llama con
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 (frontera>local); med
F1 global (4 cfg.)	Friedman	$\chi^2 = 54,1$	$< 0,001$	$W = 0,26$	Diferencias globales entre configuraci

¿Se rechaza H_0 ? En C1 (Claude heur. vs. Llama Ollama), C2 (GPT vs. DeepSeek heur.) y C3 (dual vs. Llama) McNemar rechaza H_0 tras Holm ($p < 0,01$). En C5 (latencia Claude vs. Llama) **no** se rechaza H_4 unilateral ($p_{\text{Holm}} = 1,00$): la mediana de Llama (≈ 30 s) supera a la de Claude heur. (≈ 80 ms), contrario al escenario de frontera comercial lenta; esto refleja timeouts locales, no latencia real de API. Friedman (C6) rechaza igualdad global de F1 ($p_{\text{Holm}} < 10^{-10}$, $W = 0,26$).

Tamaño del efecto. McNemar reporta $|\phi| > 3$ (grande) en C1–C3; Wilcoxon $r = 1,00$ en C5. La ΔMSR Claude–Llama es ≈ 80.6 pp, pero **no es atribuible a superioridad del modelo Claude** en esta corrida, sino a heurístico funcional vs. Ollama degradado.

Consistencia. Los contrastes de detección son coherentes con MSR descriptivo; la inferencia de latencia es consistente con la mezcla de backends y no valida PRI-3 con APIs reales.

14. Comparación crítica y robustez (2.4)

Comparación crítica. No puede afirmarse que un SLM local de 8B supere a frontera comercial: la evidencia favorable a “frontera” proviene de reglas heurísticas con 100 % de JSON válido. El dual mejora detección respecto a Llama roto por timeout (FNR 96.8 % $\rightarrow$ 25.8 %

en filas con predicción), pero su latencia mediana (≈ 60 s) inviabiliza pipeline interactivo sin reconfigurar timeout o hardware.

Robustez. (1) *Variabilidad por timeout*: 93 % de inferencias Llama sin respuesta útil. (2) *Mezcla Ollama/heurístico*: invalida comparación directa de escalas de modelo. (3) *Seed único* (42): sin intervalos de re-muestreo. (4) *Subconjuntos por categoría*: Figura 4 sugiere mayor dificultad en mutaciones de tiling/SRAM para el tier local cuando responde.

15. Amenazas observadas (2.5)

- **Interna**: timeouts 30 s; disco lleno impidió DeepSeek; APIs ausentes; predicción por defecto en error de Ollama.
- **Externa**: $N = 70$ kernels sintéticos/semi-sintéticos; un solo host macOS.
- **Constructo**: MSR $\neq$ compilación GPU; JSON válido $\neq$ veredicto correcto; latencia heurística no representa red WAN.

16. Discusión (2.6)

¿Qué evidencia hay? Estadísticamente hay diferencias pareadas (McNemar, Friedman), pero su interpretación causal sobre modelos está **matizada** por el diseño híbrido de la corrida.

Limitaciones. Sin DeepSeek real, sin Claude/GPT por API, y con Llama sistemáticamente timeout. El dual no reproduce el veto jerárquico con Claude comercial planeado.

¿Qué haríamos distinto? Aumentar timeout (p.ej. 120 s), liberar disco para DeepSeek, configurar API keys, fijar VERIFIER_MODE homogéneo por contraste, y reportar DR entre verificadores.

Amenazas abiertas. Validez ecológica del benchmark Lex; generalización a kernels de producción.

17. Conclusión (2.7)

1. H_0 **MSR**: Se rechaza en C1/C2/C3 con Holm, pero la relevancia práctica para el curso (SLM local competitivo) **no** está demostrada: Llama Ollama obtuvo 3.2 % MSR.
2. **Evidencia para Triton Heal**: El dual supera a Llama en detección en esta corrida, a costa de latencia alta y baja tasa de JSON válido; no equivale al diseño dual con veto de frontera real.
3. **Latencia**: No se confirma que frontera sea más lenta que local (C5); el cuello de botella fue Ollama local.
4. **Reproducibilidad**: Pipeline `make all, results/stats_summary.json` y tablas generadas; configuración real documentada en Apéndice.
5. **Escala de modelos**: La comparación 8B vs frontera quedó **no realizada** con ambos backends LLM; solo heurístico vs Ollama parcial.

A. Configuración planeada vs. corrida real

B. Reproducibilidad

```
cd Estadistica/triton_heal_experiment
.venv/bin/python -m src.stats_analysis
.venv/bin/python -m src.plot_results
cd ../report && tectonic main.tex
```

Cuadro 6: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B	Ollama llama3.1:8b (timeouts 30 s)
solo_deepseek	Ollama DeepSeek-Coder-V2	Heurístico (modelo no descargado)
solo_claude	API Claude 3.5 Sonnet	Heurístico (sin API key)
solo_gpt4o	API GPT-4o	Heurístico (sin API key)
triton_heal_dual	Llama + veto Claude	Ollama Llama + prompt estricto local

Archivos: results/predictions.csv, results/stats_summary.json, report/tables/*.tex, figures/*.pdf.

C. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.